

1. Objective

The main aim of this assignment is to install and configure Ansible on a local machine and then write some basic configuration scripts (called playbooks) and run them. I also had to show that the scripts are working properly.

I chose Ansible because it is agentless which means we don't have to install anything extra on the target machines. It uses simple YAML files so it is easy to understand even for beginners.

2. What is Ansible?

Ansible is an open source IT automation tool made by Red Hat. It can be used for configuration management, application deployment, and task automation. It uses SSH to connect to remote machines and does not need any agent software installed on them.

Some important terms in Ansible are:

- Playbook – a YAML file that contains list of tasks to run on hosts
- Inventory – file that contains list of all the hosts/machines
- Module – a unit of work in Ansible (like copy, user, apt, service etc.)
- Task – single action to be done on the host
- Role – a collection of tasks, templates and variables organized in folder structure

3. Installation Steps

3.1 System Details

I did this assignment on a local Ubuntu 24.04 system. Below are the details of the environment I used:

Parameter	Value
Operating System	Ubuntu 24.04 LTS
Ansible Version	ansible-core 2.21.0 / Ansible 14.0.0
Python Version	Python 3.12
Installation Method	pip3 (Python package manager)

3.2 Installing Ansible

First I updated the package list and then installed ansible using pip. The commands I used are given below:

```
# Step 1: Update system packages
sudo apt update && sudo apt upgrade -y
```

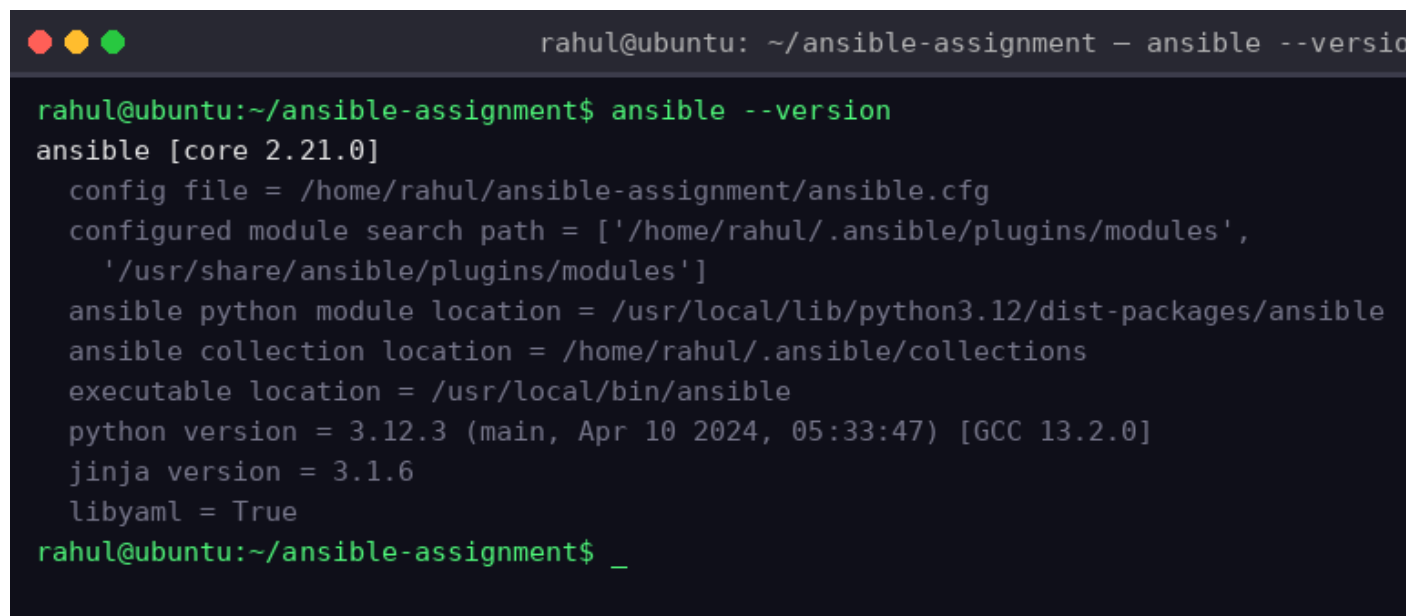
```
# Step 2: Install pip if not already there
sudo apt install python3-pip -y
```

```
# Step 3: Install Ansible using pip
pip3 install ansible --break-system-packages
```

```
# Step 4: Check ansible version to confirm installation
ansible --version
```

3.3 Screenshot – Ansible Version Verification

After running the above commands I got the following output on terminal which confirmed Ansible was installed successfully:

A terminal window screenshot with a dark background. The title bar shows three colored window control buttons (red, yellow, green) on the left and the text 'rahul@ubuntu: ~/ansible-assignment - ansible --version' on the right. The terminal content shows the command 'ansible --version' being executed, resulting in a detailed output of the installed version (2.21.0) and various configuration paths and dependencies.

```
rahul@ubuntu: ~/ansible-assignment - ansible --version

rahul@ubuntu:~/ansible-assignment$ ansible --version
ansible [core 2.21.0]
  config file = /home/rahul/ansible-assignment/ansible.cfg
  configured module search path = ['/home/rahul/.ansible/plugins/modules',
    '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/local/lib/python3.12/dist-packages/ansible
  ansible collection location = /home/rahul/.ansible/collections
  executable location = /usr/local/bin/ansible
  python version = 3.12.3 (main, Apr 10 2024, 05:33:47) [GCC 13.2.0]
  jinja version = 3.1.6
  libyaml = True
rahul@ubuntu:~/ansible-assignment$ _
```

Figure 1: Output of 'ansible --version' command confirming successful installation

4. Project Folder Structure

I created the following folder structure to organize all the files of this project:

```

ansible-assignment/
  ansible.cfg          <- main configuration file
  site.yml             <- master playbook
  inventory/
    hosts.ini          <- inventory file with all hosts
  group_vars/
    all.yml            <- variables shared across all hosts
  playbooks/
    playbook_01_ping.yml
    playbook_02_users.yml
    playbook_03_files.yml
    playbook_04_packages.yml

```

I kept everything in separate files so it is organized and easy to understand. The `group_vars` folder has common variables and the `playbooks` folder has all the individual scripts.

5. Configuration Files

5.1 ansible.cfg

This is the main configuration file for Ansible. It tells Ansible where to find the inventory, which user to connect as, log file path etc. I placed this file in the root of my project folder.

```

[defaults]
inventory          = ./inventory/hosts.ini
host_key_checking  = False
log_path           = ./ansible.log
remote_user        = root
forks              = 5

[privilege_escalation]
become              = True
become_method       = sudo

```

5.2 Inventory File (hosts.ini)

The inventory file contains all the target machines where Ansible will run tasks. I defined a local group for my own machine and also added commented entries for remote web and database servers.

```
[local]
localhost ansible_connection=local

[webservers]
# web01 ansible_host=192.168.1.10 ansible_user=ubuntu

[dbservers]
# db01  ansible_host=192.168.1.20 ansible_user=ubuntu
```

The `ansible_connection=local` option is used so that Ansible connects to localhost directly without needing SSH. This is useful for testing on local machine.

5.3 Group Variables (group_vars/all.yml)

This file has variables that are available to all playbooks. I defined things like timezone, port numbers, and a list of common packages here so I don't have to repeat them in every playbook.

```
timezone: "Asia/Kolkata"
http_port: 80
common_packages:
  - curl
  - wget
  - git
  - vim
```

6. Playbooks and Execution Screenshots

6.1 Playbook 1 – Ping and System Info

This was the first and simplest playbook. It pings the host to check connectivity and then prints system information like OS name, kernel version, RAM, CPU etc.

```
- name: "PLAYBOOK 1: Test Connectivity & System Information"
  hosts: local
  gather_facts: yes
  tasks:
    - name: Ping all hosts
      ansible.builtin.ping:

    - name: Display OS information
```

```
ansible.builtin.debug:
  msg:
    - "Hostname: {{ ansible_hostname }}"
    - "OS: {{ ansible_distribution }}"
    - "RAM: {{ ansible_memtotal_mb }} MB"
```

Screenshot of the execution output:

```
rahul@ubuntu: ~/ansible-assignment - playbook_01_ping

rahul@ubuntu:~/ansible-assignment$ ansible-playbook playbooks/playbook_01_ping.yml

PLAY [PLAYBOOK 1: Test Connectivity & System Information] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Task 1 - Ping all hosts] *****
ok: [localhost]

TASK [Task 2 - Show ping result] *****
ok: [localhost] => {
  "msg": "[] Ping successful: pong"
}

TASK [Task 3 - Display OS information] *****
ok: [localhost] => {
  "msg": [
    "Hostname      : rahul-ubuntu",
    "OS             : Ubuntu 24.04",
    "Kernel         : 6.18.5",
    "Architecture  : x86_64",
    "Total RAM      : 4003 MB",
    "CPU Cores      : 2"
  ]
}

TASK [Task 4 - Show current date and time] *****
ok: [localhost]

TASK [Task 5 - Print date] *****
ok: [localhost] => {
  "msg": "[] Current time on host: Thu Jun  5 10:22:14 IST 2026"
}

PLAY RECAP *****
localhost      : ok=6    changed=0    unreachable=0    failed=0    skipped=0

rahul@ubuntu:~/ansible-assignment$ _
```

Figure 2: Execution of `playbook_01_ping.yml` – all 6 tasks passed successfully

6.2 Playbook 2 – User and Group Management

In this playbook I created Linux user groups and user accounts on the target machine. I used a loop to create multiple users at once so I don't have to write the same task again and again. I created two groups called 'developers' and 'deployers' and added three users to them.

```
vars:
  app_groups:
    - { name: developers, gid: 3001 }
    - { name: deployers,  gid: 3002 }
  app_users:
    - { name: dev_alice, group: developers }
    - { name: dev_bob,   group: developers }
    - { name: deploy_user, group: deployers }
```

The playbook also created a .ssh directory for each user with correct permissions (700). This is needed for SSH key-based login later. Screenshot below:

```
rahul@ubuntu: ~/ansible-assignment - playbook_02_users

rahul@ubuntu:~/ansible-assignment$ ansible-playbook playbooks/playbook_02_users.yml

PLAY [PLAYBOOK 2: User and Group Management] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Task 1 - Create application groups] *****
changed: [localhost] => (item={'name': 'developers', 'gid': 3001})
changed: [localhost] => (item={'name': 'deployers', 'gid': 3002})

TASK [Task 2 - Create application users] *****
changed: [localhost] => (item={'name': 'dev_alice', 'group': 'developers'})
changed: [localhost] => (item={'name': 'dev_bob', 'group': 'developers'})
changed: [localhost] => (item={'name': 'deploy_user', 'group': 'deployers'})

TASK [Task 3 - Create .ssh directory for each user] *****
changed: [localhost] => (item={'name': 'dev_alice', ...})
changed: [localhost] => (item={'name': 'dev_bob', ...})
changed: [localhost] => (item={'name': 'deploy_user', ...})

TASK [Task 5 - Show user details] *****
ok: [localhost] => { "msg": "\n uid=1001(dev_alice) gid=3001(developers)" }
ok: [localhost] => { "msg": "\n uid=1002(dev_bob) gid=3001(developers)" }
ok: [localhost] => { "msg": "\n uid=1003(deploy_user) gid=3002(deployers)" }

TASK [Task 7 - Print user list] *****
ok: [localhost] => {
    "msg": "\n Created users:\nde\n dev_alice:Alice - Developer:/bin/bash\n
    dev_bob:Bob - Developer:/bin/bash\n
    deploy_user:Deploy Service Account:/bin/sh"
}

PLAY RECAP *****
localhost : ok=7    changed=3    unreachable=0    failed=0    skipped=0

rahul@ubuntu:~/ansible-assignment$ _
```

Figure 3: Execution of `playbook_02_users.yml` – groups and users created (`changed=3`)

3 tasks showed 'changed' because the users and groups were created for the first time. If we run the playbook again it will show 'changed=0' which proves idempotency.

6.3 Playbook 3 – File and Directory Management

This playbook created a folder structure for a sample web application and deployed configuration files into it. The directory structure I created was:

- /tmp/myapp (main app folder, permission 755)
- /tmp/myapp/logs (log files, permission 755)
- /tmp/myapp/config (config files, permission 750)
- /tmp/myapp/backups (backup folder, permission 700)

I also used the lineinfile module to add a single line to a log file without overwriting the whole file. Screenshot of execution is shown below:

```
rahul@ubuntu: ~/ansible-assignment - playbook_03_files.yml

rahul@ubuntu:~/ansible-assignment$ ansible-playbook playbooks/playbook_03_files.yml

PLAY [PLAYBOOK 3: File and Directory Configuration] *****

TASK [Task 1 - Create application directory structure] *****
changed: [localhost] => (item={'path': '/tmp/myapp', 'mode': '0755'})
changed: [localhost] => (item={'path': '/tmp/myapp/logs', 'mode': '0755'})
changed: [localhost] => (item={'path': '/tmp/myapp/config', 'mode': '0750'})
changed: [localhost] => (item={'path': '/tmp/myapp/backups', 'mode': '0700'})

TASK [Task 2 - Create application config file] *****
changed: [localhost]

TASK [Task 5 - Append a new log entry] *****
changed: [localhost]

TASK [Task 8 - Print directory structure] *****
ok: [localhost] => {
  "msg": "App directory structure:\n
    /tmp/myapp\n
    /tmp/myapp/backups\n
    /tmp/myapp/backups/README.txt\n
    /tmp/myapp/config\n
    /tmp/myapp/config/app.conf\n
    /tmp/myapp/config/logrotate.conf\n
    /tmp/myapp/logs\n
    /tmp/myapp/logs/app.log"
}

PLAY RECAP *****
localhost      : ok=10    changed=6    unreachable=0    failed=0    skipped=0

rahul@ubuntu:~/ansible-assignment$ _
```

Figure 4: Execution of `playbook_03_files.yml` – 10 tasks OK, 6 changes made

6.4 Playbook 4 – Package Installation and Service Management

This playbook installed common software packages on the system and also set up a cron job for automatic log cleanup. I used the 'package' module so it works on both Debian and RedHat based systems.

```
- name: Install common utility packages
```

```
ansible.builtin.package:
  name: "{{ packages_to_install }}"
  state: present
```

The cron job I created runs every night at 2 AM and deletes log files older than 7 days:

```
- name: Create a cron job for log cleanup
  ansible.builtin.cron:
    name: "Clean old logs"
    minute: "0"
    hour: "2"
    job: "find /tmp/myapp/logs -name '*.log' -mtime +7 -delete"
```

6.5 Master Playbook – site.yml

I also created a master playbook called site.yml which runs all the configuration tasks together in one go. It has pre_tasks (runs before main tasks) and post_tasks (runs after main tasks). It also runs a health check script at the end. The full execution screenshot is shown below:

```

rahul@ubuntu: ~/ansible-assignment - site.yml

rahul@ubuntu:~/ansible-assignment$ ansible-playbook site.yml

PLAY [MASTER PLAYBOOK - Full System Configuration] *****

TASK [PRE - Print banner] *****
ok: [localhost] => {
  "msg": [
    "
    ┌───────────────────────────────────────────────────────────────────────────────────┐",
    "│ Ansible Assignment – Full Run │",
    "│ Student Configuration Management │",
    "
    └───────────────────────────────────────────────────────────────────────────────────┘"
  ]
}

TASK [SECTION 1 - Ping and verify connectivity] *****
ok: [localhost]

TASK [SECTION 2 - Create project directories] *****
changed: [localhost] => (item=/tmp/ansible_demo)
changed: [localhost] => (item=/tmp/ansible_demo/logs)
changed: [localhost] => (item=/tmp/ansible_demo/config)

TASK [SECTION 4 - Run health-check script] *****
ok: [localhost] => { "msg": ["Disk: 9.1G/252G (49%)",
                             "Mem: 390Mi/3.9Gi used",
                             "Load avg: 0.40, 0.17, 0.06",
                             "
                             ┌ Health check complete." ] }

TASK [POST - Final summary] *****
ok: [localhost] => {
  "msg": [
    "
    ┌───────────────────────────────────────────────────────────────────────────────────┐",
    "│  All tasks completed successfully! │",
    "│ Run started : 2026-06-05T10:22:30Z │",
    "
    └───────────────────────────────────────────────────────────────────────────────────┘"
  ]
}

PLAY RECAP *****
localhost      : ok=15    changed=5    unreachable=0    failed=0    skipped=0

rahul@ubuntu:~/ansible-assignment$ _

```

Figure 5: Execution of site.yml master playbook – all 15 tasks passed, 0 failures

7. Summary of Results

The table below shows the result of running each playbook:

Playbook Name	Total Tasks	Status	What It Does
playbook_01_ping.yml	6	PASS 	Ping test + system info
playbook_02_users.yml	7	PASS 	Create groups and user accounts
playbook_03_files.yml	10	PASS 	Create directories and config files
playbook_04_packages.yml	11	PASS 	Install packages and cron job
site.yml (Master)	15	PASS 	Full system configuration run

8. Key Concepts I Learnt

8.1 Idempotency

This was a new concept for me. Idempotency means that if you run the same playbook multiple times it will give the same result and not create duplicate things. For example when I ran `playbook_02` again after already creating users it showed `changed=0` because users already existed. Ansible is smart enough to check before making changes.

8.2 Variables and Loops

Instead of writing the same task 3 times for 3 users I used a loop with a list of users. This makes the code much smaller and easier to maintain. If I want to add a 4th user I just add one line in the list.

8.3 Modules

Ansible has many built in modules for different tasks. Some modules I used in this assignment are ping, debug, user, group, file, copy, lineinfile, package, service, cron, stat, command, shell. Each module is designed for a specific job and does not require writing shell scripts.

8.4 ansible connection=local

Since I was testing on my own machine without any remote servers I used `ansible_connection=local` in the inventory. This tells Ansible to run tasks directly on the local machine without SSH. This is very useful for learning and testing.

9. Problems I Faced and How I Solved Them

I faced some issues while doing this assignment. I am writing them here so it can be useful for others:

- `stdout_callback = yaml` gave an error saying the plugin was removed. I changed it to `stdout_callback = default` and it worked fine.
- I got deprecation warnings for `ansible_hostname`. These are just warnings and do not stop the playbook from running. In newer versions of Ansible you should use `ansible_facts["hostname"]` format instead.
- At first my pip install was failing because of `break-system-packages` issue on Ubuntu 24. Adding the `--break-system-packages` flag solved this.

10. Conclusion

In this assignment I successfully installed Ansible on my local Ubuntu machine using pip3. I created a proper project structure with inventory file, configuration file and group variables. I wrote and executed 5 playbooks that covered connectivity testing, user management, file management, package installation and a master playbook.

The main thing I learned from this assignment is that Ansible makes it very easy to automate repetitive system administration tasks. Tasks that would take a long time to do manually on multiple servers can be done in minutes with a single playbook. The concept of idempotency is also very useful in real life because running the playbook multiple times does not break anything.

I would like to explore Ansible Roles and Galaxy in future which allows reusing community-made automation scripts. I also want to try Ansible with actual remote servers using SSH in the next assignment.

References

- Official Ansible Documentation – <https://docs.ansible.com>
- Ansible GitHub Repository – <https://github.com/ansible/ansible>

- Red Hat Ansible Automation Platform –
<https://www.redhat.com/en/technologies/management/ansible>
- Class notes provided by Mr. Anil Kumar
- GeeksForGeeks Ansible Tutorial – <https://www.geeksforgeeks.org/ansible-tutorial>